Campus Resource Management and Booking System

for

Universities and Corporate Campuses

Prepared by

Koyyana Ram Leela Sai	B240216CS	koyyana_b240216@nitc.ac.in
Sharad Chandra Munugala	B241357CS	sharad_b241357@nitc.ac.in
Ramidi Dheeraj Reddy	B240280CS	ramidi_b240280cs@nitc.ac.in
Gunnala Mitesh Kumar	B240187CS	gunnala_b240187@nitc.ac.in

Instructor: Professor Prabhu Mohandas

Course: CS2011E DBMS

Date: 14/03/2026

Contents

Revision History **3**

Executive Summary **4**

1 Introduction **5**
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Guide 5
1.4 Product Scope 6
1.5 Definitions, Acronyms and Abbreviations 6
1.6 References 7

2 Overall Description **8**
2.1 Product Perspective 8
2.2 Product Functions 8
2.3 User Classes and Characteristics 8
2.4 Operating Environment 9
2.5 Design and Implementation Constraints 9
2.6 Assumptions and Dependencies 9

3 External Interface Requirements **10**
3.1 User Interfaces 10
3.2 Hardware Interfaces 10
3.3 Software Interfaces 10
3.4 Communication Interfaces 10

4 System Features (Functional Requirements) **11**
4.1 User Authentication and Role Management 11
4.2 Resource Registration and Management 11
4.3 Resource Discovery and Availability Checking 11
4.4 Booking Engine 11
4.5 Priority-Based Scheduling 12
4.6 Conflict Detection and Resolution 12
4.7 Approval and Workflow Management 12
4.8 Utilization Monitoring and Analytics 12
4.9 Notification and Alert System 13
4.10 Audit Logging and Activity Tracking 13
4.11 Administrative Control Panel 13

5 Database and Data Management **14**

5.1 Entity Descriptions 14
5.2 Relationships and Cardinality 15
5.3 Database Schema Design 16
5.4 Booking Conflict Logic — Stored Procedure 17
5.5 Analytical Queries and Reports 17
5.6 Indexing and Query Optimization 18
5.7 Concurrency Control Strategy 19
5.8 Priority System Design 19

6 Non-Functional Requirements 20
6.1 Performance 20
6.2 Security 20
6.3 Usability 20
6.4 Maintainability and Scalability 20

7 System Models 21
7.1 Entity-Relationship (ER) Diagram 21
7.2 Sequence Diagram — Booking Creation Flow 21

8 Other Requirements 23
8.1 Deployment 23
8.2 Monitoring 23
8.3 Compliance 23

A Glossary 24

Revision History

Version 1.0 Mar 13, 2026
Revised: Initial phase and SRS document design.

Executive Summary

The Campus Resource Allocation System is designed to efficiently manage and allocate shared campus resources such as laboratories, meeting halls, seminar halls, classrooms, and equipment. In many institutions, these resources are often booked manually or through informal communication, which can lead to scheduling conflicts, double bookings, and inefficient utilization.

This project introduces a centralized digital platform where users can view available resources and request reservations. Administrators can manage resources, categorize them (e.g., lab, classroom, meeting room), and approve or reject booking requests. The system also supports role-based access, ensuring that different users such as administrators and regular users have appropriate permissions.

By automating the allocation process, the system improves transparency, reduces scheduling conflicts, and ensures optimal utilization of campus facilities. Overall, the Campus Resource Allocation System simplifies resource management and enhances operational efficiency within the institution.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for the Campus Resource Management and Booking System (CRMBS). It will serve as the main reference for the design, development, testing, and deployment of the system.

The main objectives of this system are:

(a) To prevent scheduling conflicts by ensuring that the same resource cannot be booked for the same time slot more than once.
(b) To provide clear information about the availability and usage of campus resources so that all users can easily see which resources are free or already booked.
(c) To support role-based access and structured booking workflows, allowing different users such as administrators and regular users to request, manage, and approve bookings in an organized way.

1.2 Document Conventions

The following conventions are used throughout this document:

- CRMBS — Campus Resource Management and Booking System.
- SHALL — mandatory requirement; non-compliance is a defect.
- SHOULD — recommended; deviation requires written justification.
- MAY — optional feature; included based on available time.
- FR-XXX — unique identifier for a functional requirement.
- HOD — Head of the Department.

1.3 Intended Audience and Reading Guide

This document is intended for different groups of readers who are involved in the development, evaluation, and use of the system.

- **Project Supervisor / Academic Evaluator:** Should review all sections of the document to understand the complete design, requirements, and functionality of the system.
- **Software Developers:** Should focus mainly on Sections 3, 4, 5, and 7, which describe the system features, functional requirements, system architecture, and implementation details.
- **Database Administrators:** Should read Sections 5, 6, and 8, which cover database design, data management, and related technical aspects.
- **Quality Assured Engineers / Testers:** Should focus on Sections 4 , which describe system requirements and testing procedures.
- **End Users and Managers:** Should mainly read Sections 2 and 4 to understand the system overview and how the system functions from a user perspective.

1.4 Product Scope

CRMBS is a multi-tier web application deployable on-premise or in the cloud. It serves organizations that need centralized management of shared physical resources. The system scope includes:

- User registration via Firebase Auth and role assignment by administrators.
- Resource catalog management: rooms, labs, equipment, and other bookable assets.
- Real-time availability checking and conflict-free booking enforced at the database level.
- Two-step approval workflows for restricted or high-demand resources.
- Priority-based scheduling with numeric priority levels per user role.
- Utilization analytics via materialized views with scheduled refresh.
- Automated in-system notifications logged to the database.

1.5 Definitions, Acronyms and Abbreviations

- **CRMBS:** Campus Resource Management and Booking System.
- **SRS:** Software Requirements Specification — a document that describes the system requirements.
- **RBAC:** Role-Based Access Control — permissions are given based on the user's role stored in the users table.
- **Firebase Auth:** Firebase Authentication service used to handle user login and authentication.
- **firebase_uid:** A unique user ID provided by Firebase for each authenticated user, stored in the users table.
- **DBMS:** Database Management System used to store and manage data (PostgreSQL).
- **FR:** Functional Requirement — a feature or function the system must perform.
- **ER Model:** Entity-Relationship model used to design the database structure.
- **3NF:** Third Normal Form — a database design rule that removes unnecessary data duplication.
- **ACID:** Properties of database transactions ensuring reliability — Atomicity, Consistency, Isolation, Durability.
- **MVCC:** Multi-Version Concurrency Control — a PostgreSQL mechanism that allows multiple users to access the database simultaneously.
- **Advisory Lock:** A PostgreSQL locking mechanism used to control concurrent booking attempts.
- **Materialized View:** A stored query result in the database used for faster analytics and reporting.
- **Audit Trigger:** A database trigger that records changes made to data in an audit log table.
- **Temporal Data:** Data that keeps track of changes over time.
- **DDL:** Data Definition Language — SQL commands used to create or modify database structures (CREATE, ALTER, DROP).
- **Two-Step Approval:** A booking approval process where a Resource Manager approves first and then a Department Manager confirms.
- **WAL:** Write-Ahead Log — a PostgreSQL feature used for data recovery.
- **REST:** Representational State Transfer — a style used for building web APIs.

- **JWT:** JSON Web Token — a secure token issued during authentication and used to verify user identity.

1.6 References

- IEEE Std Systems and Software Engineering — Requirements Engineering.
- Firebase Authentication Documentation.
- PostgreSQL Documentation.

2. Overall Description

2.1 Product Perspective

The Campus Resource Management and Booking System (CRMBS) is a new standalone system designed to replace manual booking methods such as emails and spreadsheets that are currently used to manage campus resources.

User authentication is handled by Firebase Authentication. The system does not store user passwords or login credentials locally. Instead, it only stores the user's Firebase ID and their assigned role in the PostgreSQL database.

The system may integrate with an external email service to send booking notifications to users. It also provides a calendar synchronization feature so that users can view their bookings in personal calendar applications.

2.2 Product Functions

- **Identity & Access:** The system uses Firebase Authentication to handle user login, logout, and password reset.
- **Resource Catalog:** Administrators can create, update, and deactivate resources such as labs, classrooms, meeting rooms, and equipment. Each resource includes details like capacity, location, and category.
- **Booking Engine:** Users can search for available resources, create bookings, and cancel bookings. The system prevents booking conflicts using database-level checks.
- **Scheduling Intelligence:** Each user role has a priority level. This priority is stored with the booking and helps in scheduling decisions.
- **Two-Step Approval Workflow:** Some restricted resources require two levels of approval. First, the Resource Manager approves the request, and then the Department Manager gives final approval.
- **Analytics & Reports:** The system provides reports such as resource usage statistics, bookings per department, most frequently used resources, and occupancy rates.
- **Notifications:** When booking status changes (such as approval or cancellation), the system records it and sends email notifications through background processing.
- **Audit & Compliance:** Important database operations such as insert, update, and delete are recorded in an audit log to maintain a history of changes.
- **Administration:** Administrators can manage users, assign roles, manage resources, set blackout periods, and configure system settings through the admin panel.

2.3 User Classes and Characteristics

- **System Administrator:** Has full control over the system. Responsible for managing users, assigning roles, overriding bookings, and configuring blackout periods.
  Priority: 1 (Highest). Access: Unrestricted system access.
- **Resource Manager:** Manages specific resource categories such as labs or meeting

rooms. Acts as the first approver in the two-step booking approval process.
Priority: 2. Access: Manage assigned resources and perform first-level approvals.

- **Department Manager:** Manages bookings at the department level and acts as the second and final approver in the two-step approval workflow.
  Priority: 2. Access: View department bookings and perform second-level approvals.
- **Staff / Faculty:** Regular employees who create bookings for academic or work-related activities.
  Priority: 3. Access: Can create and view their own bookings.
- **Student:** Users with the lowest priority. They may have stricter limits on booking times and the number of bookings allowed.
  Priority: 4 (Lowest). Access: Can create and view their own bookings with certain restrictions.

2.4 Operating Environment

- Server OS: Ubuntu 22.04 LTS or equivalent, containerized via Docker 24+.
- Database: PostgreSQL 15+.
- Application Tier: Node.js 20 LTS (REST API) or Python 3.11 (FastAPI).
- Authentication: Firebase Authentication (Google Firebase project required for deployment).
- Frontend: React 18 SPA; responsive design; tested on Chrome, Firefox, and Edge current versions.

2.5 Design and Implementation Constraints

- All booking conflict prevention MUST be enforced at the PostgreSQL level via stored procedure — application-level checks alone are insufficient.
- No passwords or credential data shall be stored in the local database; all authentication is handled by Firebase Auth.
- Role assignment is performed exclusively by System Administrators through the admin panel; users cannot self-assign roles.
- The audit_log table MUST be append-only; UPDATE and DELETE operations on it are blocked by database rules for all roles including admin.

2.6 Assumptions and Dependencies

- An institutional email server (SMTP) or email relay service is available for sending notifications.
- Users have access to a modern web browser; no native app installation is required.
- Public holiday and blackout calendars are maintained by a System Administrator via the admin panel.

3. External Interface Requirements

3.1 User Interfaces

- A responsive web application providing role-specific dashboards: booking calendar, list view, and analytics panels.
- A 3-step booking wizard: (1) Select resource and time, (2) Review availability, (3) Confirm.
- An admin panel with tabbed sections: Users, Resources, Bookings, Approvals, Reports, System Config.

3.2 Hardware Interfaces

CRMBS is a software-only system. Standard client devices (desktop, laptop, tablet, smartphone) with a supported browser are sufficient.

3.3 Software Interfaces

The system integrates with Firebase Authentication to handle user login and identity verification. User credentials are managed by Firebase, while the application stores only the user's Firebase ID and role information.

The system uses PostgreSQL as the database to store data related to users, resources, bookings, and system logs.

An email service (SMTP server) is used to send notifications such as booking confirmations, approvals, or cancellations to users.

The frontend application is developed using React, which communicates with the backend through REST APIs built using Node.js.

3.4 Communication Interfaces

- All communication between the client (web browser) and the server uses HTTPS to ensure secure data transfer. The system uses REST APIs that exchange data in JSON format.
- For authenticated requests, a Firebase JWT token is included in the `Authorization: Bearer` header of each API request.
- Resource availability updates are shown when the page is refreshed. The system may also check for updates automatically about every 30 seconds.
- Email notifications are sent using an SMTP email service. These notifications are processed by a background worker that checks the notification table and sends emails to users.

4. System Features (Functional Requirements)

Each subsection lists the functional requirements of the system.

4.1 User Authentication and Role Management

Authentication is handled by Firebase Authentication, and the local database stores only the `firebase_uid` and the user's application role.

- **FR-001:** The system shall authenticate users using Firebase Authentication. On the first successful login, the backend creates a new record in the users table with the firebase_uid and a default role of student.
- **FR-002:** The backend shall verify the Firebase JWT token for every protected API request.
- **FR-003:** The system shall enforce role-based access control (RBAC) based on the role stored in the users table.
- **FR-004:** A System Administrator or HOD shall be able to assign or change a user's role through the admin panel or HOD panel.
- **FR-005:** A System Administrator shall be able to deactivate a user account by setting `is_active = false`.
- **FR-006:** Password reset shall be handled entirely by Firebase Authentication.

4.2 Resource Registration and Management

- **FR-007:** A HOD shall be able to create a resource with fields including name, category, capacity, location, status, and features.
- **FR-008:** All resources shall appear in search results regardless of their status. Resources marked inactive or under maintenance cannot be booked.
- **FR-009:** Each resource shall be assigned to a category such as Classroom, Lab, Meeting Room, or Equipment. The category field is mandatory.
- **FR-010:** A Resource Manager shall be able to define recurring unavailability periods for a resource.

4.3 Resource Discovery and Availability Checking

- **FR-011:** The system shall provide a search feature allowing users to filter resources by category, minimum capacity, date, start time, and end time.
- **FR-012:** Resource availability information shall be updated when the page is refreshed.
- **FR-013:** The system shall provide a read-only API endpoint that returns the available and busy time slots for a resource on a specific day.

4.4 Booking Engine

- **FR-014:** Authenticated users shall be able to create bookings by specifying the resource, date, start time, end time, and purpose.
- **FR-015:** The system shall prevent double bookings for the same resource and time

period.

- **FR-016:** The system shall support recurring bookings with weekly frequency, where each occurrence is stored as a separate booking record.
- **FR-017:** A user shall be able to cancel their own booking before the booking start time.
- **FR-018:** Each resource category shall have a configurable maximum advance booking window.

4.5 Priority-Based Scheduling

- **FR-019:** Each user role shall have a numeric priority level stored in the users table.
- **FR-020:** The priority level shall influence scheduling decisions when administrators resolve booking conflicts.
- **FR-021:** The system shall display the priority level of each booking in the admin, HOD, and resource manager views.

4.6 Conflict Detection and Resolution

- **FR-022:** Conflict detection shall be implemented in PostgreSQL using advisory locks to prevent concurrent booking conflicts.
- **FR-023:** When a conflict occurs, the system shall suggest alternative available time slots for the same resource.
- **FR-024:** A System Administrator shall be able to cancel conflicting bookings and reassign users to alternative resources of the same category.

4.7 Approval and Workflow Management

Some resources require a two-step approval process involving a Resource Manager and a Department Manager.

- **FR-025:** Bookings for restricted resources shall be created with a status of Pending and a notification sent to the Resource Manager.
- **FR-026:** The Resource Manager shall be able to approve or reject a booking at the first approval step.
- **FR-027:** The Department Manager shall be able to approve or reject the booking at the second approval step.
- **FR-028:** Each approval step shall have a configurable deadline for approval actions.
- **FR-029:** Rejection of a booking shall require remarks explaining the reason.

4.8 Utilization Monitoring and Analytics

- **FR-030:** The system shall provide an analytics dashboard showing resource usage statistics such as bookings per week, occupancy rate, peak hours, and no-show rate.
- **FR-031:** The system shall provide a report showing bookings per department for a selected date range and allow export as CSV.
- **FR-032:** The system shall refresh analytics data using materialized views.

4.9 Notification and Alert System

- **FR-033:** The system shall create a notification record whenever a booking status changes.
- **FR-034:** A background job shall process notifications and send emails through an SMTP server.
- **FR-035:** The system shall generate reminder notifications before the start time of approved bookings.

4.10 Audit Logging and Activity Tracking

- **FR-036:** Every insert, update, and delete operation on key tables such as bookings, resources, and users shall be recorded in an audit log using database triggers.
- **FR-037:** The audit_log table shall be append-only, and update or delete operations on it shall not be allowed.
- **FR-038:** Audit logs shall be retained in the database for a minimum period of two years.

4.11 Administrative Control Panel

- **FR-039:** The system shall provide an administrative control panel for managing users, assigning roles, managing resources, viewing bookings, configuring blackout periods, and adjusting system settings.
- **FR-040:** The system shall support administrator-defined blackout periods during which new bookings cannot be created.

5. Database and Data Management

This section describes the database design used in the Campus Resource Management and Booking System (CRMBS). The system uses a relational database to store and manage all application data. The database design focuses on data accuracy, preventing booking conflicts, maintaining audit records, and ensuring efficient query performance.

The schema is designed following Third Normal Form (3NF) to reduce redundancy and maintain data consistency. Lookup tables are used for values such as booking status, resource category, and priority. The resource table includes a JSONB features field, which allows storing flexible attributes without changing the database schema.

The complete database structure, including table creation scripts, stored procedures, triggers, and queries, is maintained in the project SQL file (`schema.sql`).

5.1 Entity Descriptions

The core database schema consists of the following main entities:

- **Department Table:**
  Stores department information such as department name, building location, and contact email. This table helps organize users by department.
- **Users Table:**
  Stores user information including first name, last name, email, phone number, role, and account status. It also stores the firebase_uid, which links the user account to Firebase Authentication.
- **Resource Category Table:**
  Stores categories of resources such as classrooms, laboratories, meeting rooms, or equipment. It also defines booking policies such as advance booking limits.
- **Resource Table:**
  Stores details of individual resources including resource name, capacity, location, status, and additional features. The features are stored in JSONB format to allow flexible attributes such as projector availability or whiteboard.
- **Booking Status Table:**
  A lookup table that defines the possible booking states such as Pending, Approved, Rejected, Cancelled, and Completed.
- **Priority Table:**
  Stores priority levels assigned to different user roles. These priority levels are used during booking scheduling and conflict resolution.
- **Recurrence Table:**
  Stores information related to recurring bookings, such as weekly booking patterns, and links multiple booking occurrences together.
- **Booking Table:**
  This is the central table that stores all booking information, including booking date, start time, end time, purpose, priority level, user, resource, and booking status.
- **Approval Table:**
  Stores approval workflow information. Each booking requiring approval will have

approval records indicating which user approved or rejected the request and at which step.

- **Log Table:**
  Stores application-level activity logs that record important system events for user-visible history.
- **Audit Log Table:**
  Stores a permanent history of all database changes. This table captures insert, update, and delete operations using database triggers.
- **Notification Table:**
  Stores notification messages that are sent to users. These notifications may be delivered through email or other channels by background processing jobs.

5.2 Relationships and Cardinality

This section explains the relationships between the entities in the database along with their cardinality ratios and participation type.

- **Department — Specifies — User**
  - Cardinality: 1 : N
  - Participation: Department (Partial) / User (Total)
  - Explanation: One department can have many users, but each user must belong to exactly one department.
- **User — Places — Booking**
  - Cardinality: 1 : N
  - Participation: User (Partial) / Booking (Total)
  - Explanation: A user may create multiple bookings, but every booking must have exactly one user as its owner.
- **Resource — Reserved In — Booking**
  - Cardinality: 1 : N
  - Participation: Resource (Partial) / Booking (Total)
  - Explanation: A resource can appear in many bookings, but every booking must reference exactly one resource.
- **Resource Category — Classifies — Resource**
  - Cardinality: 1 : N
  - Participation: Category (Partial) / Resource (Total)
  - Explanation: A category can contain many resources, but every resource must belong to exactly one category.
- **Booking Status — Has Status — Booking**
  - Cardinality: 1 : 1
  - Participation: Status (Partial) / Booking (Total)
  - Explanation: Every booking must always have one status such as Pending, Approved, Rejected, Cancelled, or Completed.
- **Booking — Has — Approval**
  - Cardinality: 1 : 1 (maximum 2 approval rows)
  - Participation: Booking (Partial) / Approval (Total)
  - Explanation: Some bookings require approval. Each approval record must reference a booking, but not all bookings require approval.

- **User — Approves — Approval**
  - Cardinality: 1 : N
  - Participation: User (Partial) / Approval (Total)
  - Explanation: One user (such as a Resource Manager or Department Manager) can approve multiple bookings, and each approval record must record the approver.
- **Booking — Generates — Log**
  - Cardinality: 1 : N
  - Participation: Booking (Partial) / Log (Total)
  - Explanation: A booking can generate multiple log entries, but each log entry must reference a specific booking.
- **User — Recorded In — Log**
  - Cardinality: 1 : N
  - Participation: User (Partial) / Log (Total)
  - Explanation: Each log entry records the user who performed the action in the system.
- **User — Receives — Notification**
  - Cardinality: 1 : N
  - Participation: User (Partial) / Notification (Total)
  - Explanation: A user can receive multiple notifications, but each notification must be associated with one specific user.
- **Priority — Assigned To — Booking**
  - Cardinality: 1 : N
  - Participation: Priority (Partial) / Booking (Partial)
  - Explanation: A priority level may be assigned to bookings to influence scheduling decisions. Some bookings may use a default priority.
- **Recurrence — Groups — Booking**
  - Cardinality: 1 : N
  - Participation: Recurrence (Partial) / Booking (Partial)
  - Explanation: Recurrence groups multiple bookings together when they are repeated (for example, weekly bookings). Single bookings may not have a recurrence reference.

5.3 Database Schema Design

Overview

The schema uses PostgreSQL 15 and is organized into three logical tiers: (1) reference/lookup tables (department, resource_category, booking_status, priority) which rarely change; (2) core transactional tables (users, resource, booking, approval) which are the subject of most reads and writes; and (3) operational tables (audit_log, notification, log, recurrence) which support system behaviour and compliance.

Key schema decisions

- The users table stores `firebase_uid` (VARCHAR) as the identity bridge to Firebase Auth. No password hash is stored. The role column (VARCHAR with CHECK constraint) drives all RBAC decisions.
- The users table stores a firebase_uid which links the database user with Firebase

Authentication.
- Passwords are not stored in the database.
- The booking table uses a PostgreSQL exclusion constraint (requires btree_gist extension) on (`resource_id`, `tsrange(date+start_time, date+end_time)`) to enforce no-overlap at the database level — independent of application logic. This is the primary technical contribution of the schema design.

5.4 Booking Conflict Logic — Stored Procedure

Purpose

A PostgreSQL stored procedure (`create_booking`) implements the complete booking creation flow as a single atomic database operation. This is the most important DBMS mechanism in the project — it ensures that concurrent booking requests cannot bypass conflict detection through race conditions.

Logic flow

- **Step 1 — Input validation:** verify start_time < end_time; verify booking date does not exceed the category's advance_days window.
- **Step 2 — Is Active check:** query Resource table for the resource's category and the requested date. Raise exception if within a blackout period.
- **Step 3 — Advisory lock:** call `pg_advisory_xact_lock(resource_id)` to serialize all concurrent booking attempts for the same resource. The lock is held for the duration of the transaction and released automatically on commit or rollback.
- **Step 4 — Conflict query:** SELECT COUNT(*) from booking where resource_id matches, date matches, status is Pending or Approved, and the time ranges overlap using the condition: existing.start_time < p_end AND existing.end_time > p_start. If count > 0, raise a conflict exception.
- **Step 5 — Status assignment:** if the resource category has requires_approval = true, set status = 'Pending'; otherwise set status = 'Approved'.
- **Step 6:** Insert booking row with all validated values; copy priority_level from user record.
- **Step 7:** Insert notification row in the same transaction for booking confirmation.

5.5 Analytical Queries and Reports

Reports in the system are generated using SQL aggregation operations such as COUNT, GROUP BY, and ORDER BY. These queries help analyze system usage, detect conflicts, and produce administrative reports.

Key Analytical Queries

1. **Conflict Detection**
   - Purpose: Identify all active bookings for a specific resource on a given date where the requested time slot overlaps with existing bookings.
   - Technique / Tables: booking JOIN booking_status with time-range comparison conditions.
2. **Available Resources**
   - Purpose: Retrieve all resources within a category that do not have conflicting bookings for a selected date and time slot.

- Technique / Tables: resource LEFT JOIN booking using NOT IN subquery or LEFT JOIN + IS NULL.

3. **Busiest Resources**
   - Purpose: Determine which resources are used most frequently within a specified date range.
   - Technique / Tables: booking JOIN resource, grouped by resource_id and sorted using ORDER BY COUNT DESC.
4. **Bookings per Department**
   - Purpose: Count the number of bookings made by each department during a selected time period.
   - Technique / Tables: booking JOIN users JOIN department, grouped by dept_name.
5. **Utilization Rate**
   - Purpose: Calculate the percentage of time a resource is booked compared to its total available time within a week.
   - Technique / Tables: Uses the mv_resource_utilization materialized view with the formula ROUND((booked_hours / available_hours) * 100).
6. **Audit History for a Booking**
   - Purpose: Retrieve the complete change history of a specific booking, showing all updates and modifications.
   - Technique / Tables: Query on audit_log filtering table_name = 'booking' and matching booking_id, ordered by timestamp.

5.6 Indexing and Query Optimization

To improve query performance, indexes are created on columns that are frequently used in WHERE clauses, JOIN conditions, and ORDER BY operations. In addition to the default primary key indexes, the following indexes are implemented.

Index Strategy

1. **idx_booking_resource_time**
   - Type: B-tree (Composite)
   - Table / Columns: booking (resource_id, date, start_time, end_time)
   - Justification: Used in the main conflict-detection query where all four columns appear in the WHERE condition.
2. **idx_booking_user**
   - Type: B-tree
   - Table / Column: booking (user_id)
   - Justification: Optimizes queries that retrieve a user's booking history.
3. **idx_booking_status**
   - Type: B-tree
   - Table / Column: booking (status_id)
   - Justification: Improves filtering of bookings based on status such as Pending or Approved.
4. **idx_resource_category**

- Type: B-tree
- Table / Column: resource (category_id)
- Justification: Improves search performance when filtering resources by category.

5. **idx_audit_table_time**
   - Type: B-tree (Composite)
   - Table / Columns: audit_log (table_name, changed_at)
   - Justification: Optimizes audit history queries that filter by table name and time range.
6. **Unique Index on Materialized View**
   - Type: B-tree (Composite)
   - Table / Columns: mv_resource_utilization (resource_id, week_start)
   - Justification: Required for CONCURRENT refresh of the materialized view used for analytics dashboards.

5.7 Concurrency Control Strategy

- **Double-booking race condition:** `pg_advisory_xact_lock(resource_id)` inside the create_booking stored procedure. Serializes all booking attempts for a given resource without table-level locks; automatically released on transaction end.
- **Read queries (availability search):** Default READ COMMITTED isolation (PostgreSQL MVCC). Sufficient for read-only queries; MVCC provides consistent snapshots without blocking writers.
- **2-step approval updates:** Optimistic approach — check current status before updating; return error if status has changed. Approval steps are low-concurrency; full locking not warranted.
- **Materialized view refresh:** REFRESH MATERIALIZED VIEW CONCURRENTLY. Non-blocking refresh; read queries continue during refresh; requires unique index.
- **Bundle/multi-resource (future):** Single BEGIN…COMMIT transaction wrapping all inserts. ACID atomicity provides all-or-nothing guarantee; out of scope for v1.0.

5.8 Priority System Design

Priority in CRMBS is a numeric value (1 = highest, 4 = lowest) assigned to each user role and stored in the users table. When a booking is created, the user's current priority_level is copied to the booking row. This design serves two purposes: it provides a permanent record of the booking's priority at creation time, and it gives administrators a clear, sortable field for manual conflict resolution decisions.

- **System Administrator — Priority 1:** Highest — admin-created bookings take precedence in any manual conflict resolution.
- **Resource Manager / Dept Manager — Priority 2:** High — managerial bookings preserved over staff/student bookings when resources are scarce.
- **Staff / Faculty — Priority 3:** Standard — typical booking priority for operational staff.
- **Student — Priority 4:** Lowest — student bookings are superseded if a higher-priority user requires the same slot.

6. Non-Functional Requirements

6.1 Performance

The system is designed to provide fast and efficient responses when users search resources or create bookings. Database operations are optimized using indexes and stored procedures to ensure quick execution.

Resource availability searches are performed using optimized queries so that users can quickly find suitable resources based on filters such as category, capacity, date, and time.

The analytics dashboard uses precomputed data to load reports faster and reduce database load.

The system also supports multiple users accessing the application at the same time without significant performance issues.

Background processing is used for tasks like sending notifications and updating reports so that these tasks do not slow down the main application operations.

6.2 Security

- All credential management (password storage, hashing, reset flow) delegated to Firebase Auth — no local credential data.
- Role checked from PostgreSQL users table on every request — Firebase token alone cannot grant elevated access.
- SQL injection prevented by parameterized queries / prepared statements throughout.
- audit_log is append-only; modifications blocked by PostgreSQL rules for all roles.
- Rate limiting: 20 requests/minute per IP on all API endpoints; 5 login attempts/minute on Firebase Auth.

6.3 Usability

- New user task completion target: a first-time user SHALL be able to complete a booking within 3 minutes without prior training.
- Error messages SHALL be plain-language, action-oriented, and include a suggested resolution.
- Calendar based booking interaction.

6.4 Maintainability and Scalability

- Modular architecture: Authentication, Booking, Approval, Analytics, and Admin modules are independently deployable.
- Audit logs SHALL be retained in the primary database for a minimum of 2 years.

7. System Models

7.1 Entity-Relationship (ER) Diagram

The ER diagram presents all 12 entities from Section 5.1 in Chen notation with: attributes (PK underlined), relationship diamonds with cardinality (1:N), and participation lines (double line = total participation, single line = partial participation) as specified in Section 5.2. The corrected diagram reflects the final schema including firebase_uid on users, step_number on approval.

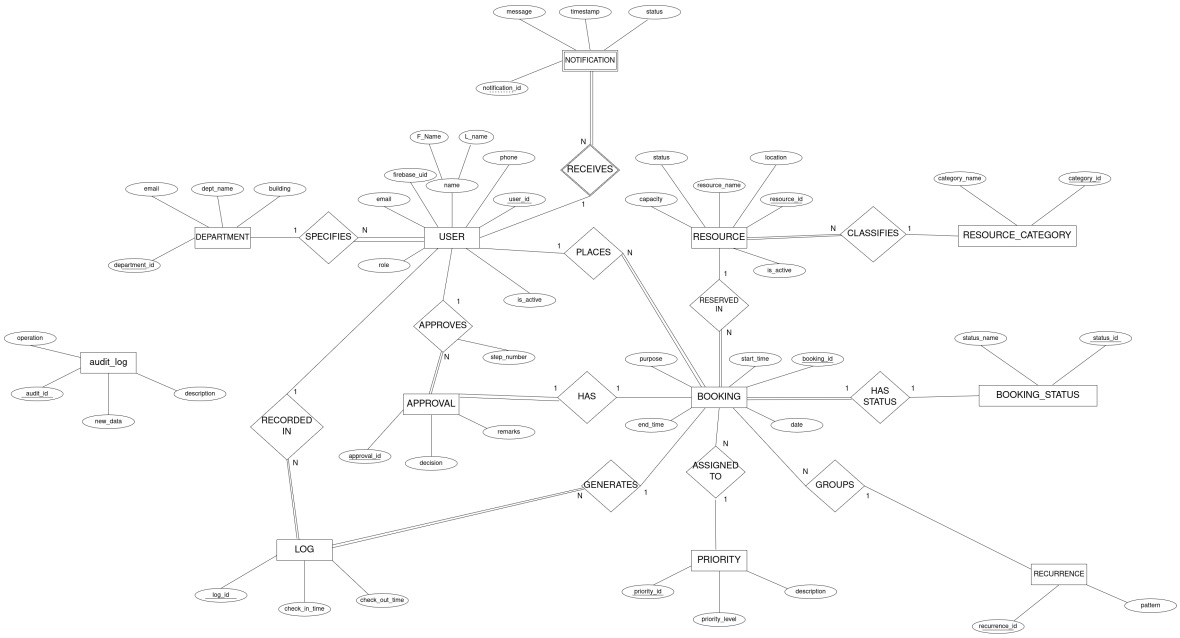

Figure 7.1: Entity-Relationship Diagram

7.2 Sequence Diagram — Booking Creation Flow

The following sequence describes the end-to-end flow for a booking that requires approval:

- 1. User submits booking form → REST POST /api/v1/bookings with Firebase JWT in header.
- 2. API middleware verifies JWT with Firebase Admin SDK → extracts firebase_uid.
- 3. Middleware queries users table → confirms user is active and retrieves role.
- 4. API calls create_booking(user_id, resource_id, date, start, end, purpose) stored procedure.
- 5. Stored procedure: acquires advisory lock → checks blackout → checks conflicts → inserts booking (status=Pending) → inserts notification row → releases lock on commit.
- 6. API returns HTTP 201 Created with booking_id and status=Pending.
- 7. Background notification job reads pending notification rows → sends email to user confirming booking is pending approval.

- 8. Resource Manager receives notification → logs into system → approves step 1.
- 9. Dept Manager notified → approves step 2 → booking status changes to Approved → user notified.

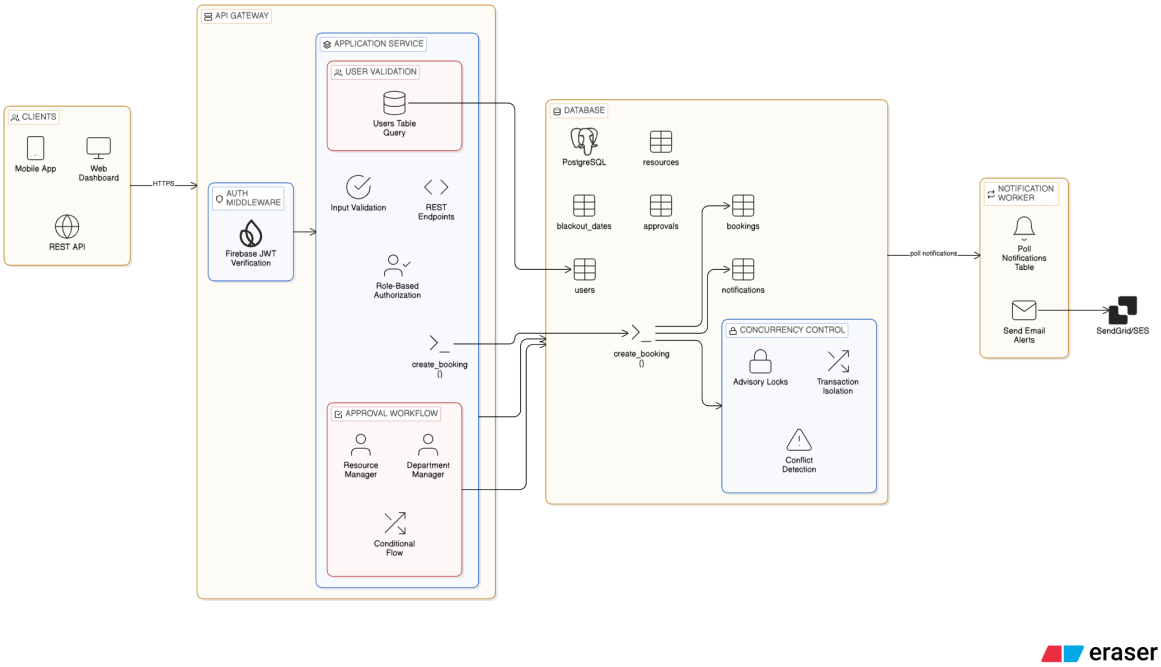

Figure 7.2: Enter Caption

Figure 7.3: System Architecture and Booking Creation Flow

8. Other Requirements

8.1 Deployment

- Docker + Docker Compose on a single Linux server.
- PostgreSQL 15+ with `btree_gist` extension.
- Firebase credentials via environment variables.

8.2 Monitoring

- Errors logged with timestamp and request context.
- Slow queries (> 200 ms) logged by PostgreSQL.

8.3 Compliance

- Audit log is append-only; no role can modify or delete records.
- Admin can anonymize user data on request.

A. Glossary

See Section 1.5 for primary definitions. Additional database-specific terms:

Term	**Definition**
Exclusion constraint	PostgreSQL constraint preventing rows from overlapping on a specified condition. Used on the booking table to enforce no-overlap of time ranges for the same resource. Requires btree_gist extension.
btree_gist	PostgreSQL extension enabling GiST indexes on scalar types; required for range-based exclusion constraints on the booking table.
Advisory lock (pg_advisory_xact_lock)	Application-level transaction lock in PostgreSQL. Serializes concurrent operations without table-level locking. Released automatically at transaction end.
Materialized view	Precomputed, stored query result in PostgreSQL. Refreshed on a schedule. The CONCURRENTLY option allows refreshes without blocking ongoing reads.
WAL (Write-Ahead Log)	PostgreSQL mechanism recording all changes before they are applied to data files. Enables crash recovery and point-in-time recovery (PITR).
GIN index	Generalized Inverted Index — used for JSONB columns to enable efficient key/value containment queries (e.g., `WHERE features @> {'has_projector': true}`).
firebase_uid	Immutable unique identifier string generated by Firebase for each user account. Used as the identity bridge between Firebase Auth and the local PostgreSQL users table.
step_number	Column in the approval table indicating whether the row represents step 1 (Resource Manager) or step 2 (Dept Manager) of the 2-step approval workflow.
fn_audit_trigger	PL/pgSQL trigger function that captures JSONB before/after snapshots of every data change into the audit_log table.